🔌 Inicialización y Auto-Instrumentación

Concepto Principal

Antes de iniciar el registro y la recolección de métricas a través de los diversos componentes (como el decorador o el middleware), es fundamental inicializar la conexión con el proveedor de observabilidad.

La función `init_telemetry` establece el vínculo entre la aplicación y **Azure Monitor** o un **Gateway OTLP**, encargándose de inyectar correctamente la cadena de conexión o endpoint y la telemetría base de OpenTelemetry. Esta configuración **solo debe ejecutarse una vez** al inicio de la aplicación.

Patrón Singleton para la Inicialización

Para garantizar que la inicialización ocurra estrictamente una única vez, se recomienda implementar un patrón **Singleton**. Esto centraliza el estado de la conexión en un módulo específico y previene instancias múltiples del trazador.

Implementación (`telemetry_singleton.py`)

A continuación, se presentan dos ejemplos de la implementación recomendada según el destino configurado para tu telemetría. En ambos modelos destaca la inclusión de `_enable_framework_auto_spans()`, función encargada de activar la **Auto-instrumentación**. La auto-instrumentación permite capturar eventos generados internamente por frameworks soportados (ej. Microsoft Agent Framework) sin tener que instanciar trazados de forma manual.

Opción A: Inicialización para Gateway OTLP

Utiliza esta configuración si envías los datos hacia un servidor OTLP (asegúrate de instalar la librería sin dependencias adicionales de Azure).

```
from azure.core.settings import settings
settings.tracing_implementation = "opentelemetry"

import os
import logging
from dotenv import load_dotenv

from monitoria.tracing.telemetry import init_telemetry

load_dotenv()

log = logging.getLogger(__name__)

tracer = None
session_id = None
server_host = None
_initialized = False
```

```
def _enable_framework_auto_spans() -> None:
    """
    Enable automatic spans for frameworks that support their own OTel hooks.
    Keep this separate from custom decorators.
    """
    # Microsoft Agent Framework
    try:
        from agent_framework.observability import enable_instrumentation

        enable_instrumentation(
            enable_sensitive_data=os.getenv(
                "ENABLE_SENSITIVE_DATA",
                "false"
            ).lower() in ("1", "true", "yes")
        )

        log.info("Microsoft Agent Framework auto-instrumentation enabled.")
    except ImportError:
        log.debug("agent_framework is not installed; skipping Agent Framework auto
spans.")
    except Exception as e:
        log.warning("Failed to enable Agent Framework auto spans: %s", e)


def setup_init_telemetry():
    global tracer, session_id, server_host, _initialized

    if _initialized:
        return tracer, session_id, server_host

    os.environ.setdefault("OTEL_SERVICE_NAME", os.getenv("OTEL_SERVICE_NAME",
"poc--dispatcher-agentes-credicorp"))
    os.environ.setdefault("ENABLE_INSTRUMENTATION", "true")
    os.environ.setdefault("ENABLE_SENSITIVE_DATA",
os.getenv("ENABLE_SENSITIVE_DATA", "false"))

    otlp_endpoint = os.getenv("OTEL_EXPORTER_OTLP_ENDPOINT",
"https://wapceu2aiasd00-f4euagbbbqdwfwg9.eastus2-01.azurewebsites.net")

    tracer, session_id, server_host = init_telemetry(
        otlp_endpoint=otlp_endpoint,
        service_name=os.environ["OTEL_SERVICE_NAME"],

capture_message_content=os.getenv("OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONT
ENT","false").lower() in ("1", "true", "yes"),
        trace_to_console=os.getenv("OTEL_TRACE_TO_CONSOLE","false").lower() in
("1", "true", "yes")
    )

    _enable_framework_auto_spans()

    _initialized = True
    return tracer, session_id, server_host
```

Opción B: Inicialización para Azure Monitor (Application Insights)

Utiliza esta configuración si fuerzas el envío a Application Insights configurando explícitamente tú la cadena de conexión (asegúrate de usar la librería instalada como `monitoria[azure-monitor]`).

```
from azure.core.settings import settings
settings.tracing_implementation = "opentelemetry"

import os
import logging
from dotenv import load_dotenv
import time

from monitoria.tracing.telemetry import init_telemetry

load_dotenv()

log = logging.getLogger(__name__)

tracer = None
session_id = None
server_host = None
_initialized = False

# 1. Tu cadena de conexión a Application Insights. Reemplázala con tu valor real.
# Normalmente se obtiene desde el portal de Azure > Application Insights >
Resumen.
AZURE_CONNECTION_STRING = "InstrumentationKey=45c8513c-b045-4478-8167-
c56790bc1664;IngestionEndpoint=https://eastus2-
3.in.applicationinsights.azure.com/;LiveEndpoint=https://eastus2.livediagnostics.m
onitor.azure.com/;ApplicationId=695076db-3b17-4ec6-a265-5a7769e1239d"

# 2. Importante: Limpiamos la variable de entorno del OTLP endpoint si estuviera
# configurada, para obligar al modulo a irse por la ruta de Azure Monitor.
if "OTEL_EXPORTER_OTLP_ENDPOINT" in os.environ:
    del os.environ["OTEL_EXPORTER_OTLP_ENDPOINT"]

def _enable_framework_auto_spans() -> None:
    """
    Enable automatic spans for frameworks that support their own OTel hooks.
    Keep this separate from custom decorators.
    """
    # Microsoft Agent Framework
    try:
        from agent_framework.observability import enable_instrumentation

        enable_instrumentation(
            enable_sensitive_data=os.getenv(
                "ENABLE_SENSITIVE_DATA",
                "false"
```

```
            ).lower() in ("1", "true", "yes")
        )

        log.info("Microsoft Agent Framework auto-instrumentation enabled.")
    except ImportError:
        log.debug("agent_framework is not installed; skipping Agent Framework auto
spans.")
    except Exception as e:
        log.warning("Failed to enable Agent Framework auto spans: %s", e)

def setup_init_telemetry():
    global tracer, session_id, server_host, _initialized

    if _initialized:
        return tracer, session_id, server_host

    os.environ.setdefault("OTEL_SERVICE_NAME", os.getenv("OTEL_SERVICE_NAME",
"poc--dispatcher-agentes-credicorp"))
    os.environ.setdefault("ENABLE_INSTRUMENTATION", "true")
    os.environ.setdefault("ENABLE_SENSITIVE_DATA",
os.getenv("ENABLE_SENSITIVE_DATA", "false"))

    print("Inicializando telemetría hacia Azure Monitor (Application
Insights)...")

    # Inicializamos la telemetría usando la cadena de conexión de App Insights.
    tracer, session_id, server_host = init_telemetry(
        appinsights_connection_string=AZURE_CONNECTION_STRING,
        service_name=os.environ["OTEL_SERVICE_NAME"],

capture_message_content=os.getenv("OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONT
ENT","false").lower() in ("1", "true", "yes"),
        trace_to_console=os.getenv("OTEL_TRACE_TO_CONSOLE","false").lower() in
("1", "true", "yes")
    )

    _enable_framework_auto_spans()

    _initialized = True
    return tracer, session_id, server_host
```

Integración en la Aplicación

Una vez configurado, cualquier otro módulo en la aplicación puede requerir las variables de telemetría llamando al Singleton:

```
from app import telemetry_singleton as telemetry

# Inicializa o devuelve la instancia si ya fue activado
telemetry.setup_init_telemetry()
```

```
# Reutilizar las variables globales del trazador
tracer = telemetry.tracer
server_host = telemetry.server_host
```

Notas sobre la configuración

- Abstenerse de ejecutar `init_telemetry` de forma independiente a lo largo del código. Utilice la función `setup_init_telemetry` del Singleton.
- Es crucial asegurarse de que las credenciales de Azure Monitor o el endpoint OTLP (`OTEL_EXPORTER_OTLP_ENDPOINT`) según corresponda estén configuradas en las variables de entorno.
- El identificador `session_id` servirá como contexto base de todo el ciclo de ejecución de la instancia.